

PROGRAMMING IN HASKELL

1



Topic 14.2 - Property-Based Testing with QuickCheck

What they are, why they matter, and how Haskell encodes them

From Unit Tests to Properties

2

Unit Tests

Specific, hand-chosen input/output pairs.

You decide which cases to test. Cases you don't write are not tested.

Example:

```
reverse [1,2,3] == [3,2,1]  
reverse []    == []
```

Properties

Universal statements true for all valid inputs.

QuickCheck generates hundreds of random inputs and checks the statement holds for every one.

Example:

```
 $\forall$  xs. reverse (reverse xs) == xs
```

Properties are stronger — they test the general case, not just the examples you thought of.

The QuickCheck Idea

3

QuickCheck turns a property into a runnable test by generating random inputs automatically.

Workflow

1. Write a property (returns Bool or Property)
2. QuickCheck generates random inputs
3. Property is checked for each input
4. Pass → reports number of tests run
5. Fail → shrinks to smallest counterexample

On success

```
+++ OK, passed 100 tests.
```

On failure

```
*** Failed! Falsified (after 47 tests):  
[-1]
```

Anatomy of a Property

4

A property is an ordinary Haskell function whose return type is `Bool` or `Property`.

```
prop_reverseInvolution :: [Int] -> Bool
prop_reverseInvolution xs =
  reverse (reverse xs) == xs
```

Running it

```
quickCheck prop_reverseInvolution
-- +++ OK, passed 100 tests.
```

Key points

- `prop_` prefix: discovered automatically by `quickCheckAll`
- Arguments are generated values (any Arbitrary type)
- Returns `Bool`: checked for ~100 generated inputs

Writing Your First Property

5

Good properties express universal truths. Three reliable patterns to reach for first.

Round-trip

Encode then decode returns the original.

```
prop_roundTrip :: String ->
Bool
prop_roundTrip s =
    decode (encode s) == Just s
```

Idempotence

Applying an operation twice gives the same result as once.

```
prop_sortIdem :: [Int] ->
Bool
prop_sortIdem xs =
    sort (sort xs) == sort xs
```

Involution

Applying the operation twice returns the original.

```
prop_reverseInv :: [Int] ->
Bool
prop_reverseInv xs =
    reverse (reverse xs) == xs
```

Generators: Arbitrary and Gen

6

QuickCheck uses Arbitrary to produce random inputs. You can customise or combine generators for any type.

Built-in Arbitrary

Standard types have instances for free.

```
arbitrary :: Gen Int
arbitrary :: Gen [Bool]
arbitrary :: Gen (Maybe
String)
```

Gen combinators

Compose primitives to build the distribution you need.

```
choose (1, 100)  :: Gen Int
elements ["a","b","c"]
oneof [genSmall, genLarge]
suchThat arbitrary (> 0)
```

Custom Arbitrary

Derive Arbitrary for your own types.

```
instance Arbitrary Day where
  arbitrary =
    Day <$> choose (1, 31)
  shrink (Day n) =
    [Day m | m <- shrink n, m >= 1]
```

Shrinking

7

When a property fails, QuickCheck shrinks the input to the smallest version that still fails.

Before shrinking

QuickCheck finds a failure on test x (e.g. 47):

```
xs = [43, -1, 12, 0, 99, 7, -5, 3]
```

After shrinking

QuickCheck reports the smallest failing case:

```
xs = [-1]
```

Why shrinking matters

A minimal counterexample is far easier to debug than a large random one. The shrink function is part of the Arbitrary instance and works recursively — QuickCheck keeps shrinking as long as the property keeps failing.

Testing Typeclass Laws

8

Typeclass laws (from previous slide deck) are exactly the kind of universal statement QuickCheck is designed to check. Write them as properties.

Functor law

```
prop_functorId :: [Int] ->
Bool
prop_functorId xs =
  fmap id xs == xs
```

Monoid law

```
prop_monoidL :: [Int] -> Bool
prop_monoidL xs =
  mempty <> xs == xs
```

Monad law

```
prop_monadL :: Int -> Bool
prop_monadL n =
  (return n >>= Just) == Just n
```

Using checkers and hspec-laws

Libraries like checkers (Test.QuickCheck.Classes) and hspec-laws bundle complete law suites for standard typeclasses. A single function call checks all Functor, Foldable, or Monad laws for your type without writing each property by hand.

Testing Smart Constructors

9

From previous slide deck : smart constructors enforce invariants at creation time. QuickCheck lets us verify this claim automatically for any generated input.

Valid inputs succeed

```
prop_mkDayValid :: Property
prop_mkDayValid =
  forAll (choose (1, 31)) $ \n ->
    isJust (mkDay n)
```

Invalid inputs fail

```
prop_mkDayInvalid :: Property
prop_mkDayInvalid =
  forAll (arbitrary `suchThat`
    (\n -> n < 1 || n > 31)) $ \n ->
    isNothing (mkDay n)
```

Round-trip invariant

```
-- Construction then extraction returns the original value
prop_roundTrip :: Property
prop_roundTrip =
  forAll (choose (1, 31)) $ \n ->
    fmap getDay (mkDay n) == Just n
```

Conditional Properties

10

Use `==>` (implies) to restrict the inputs a property applies to. QuickCheck skips inputs where the precondition is false.

Division identity with `==>`

```
prop_divMod :: Int -> Int -> Property
prop_divMod x y =
  y /= 0 ==> -- no division by zero
    (x `div` y) * y + (x `mod` y) == x
```

Same property using `forAll`

```
prop_divMod' :: Int -> Property
prop_divMod' x =
  forAll nonZero $ \y ->
    (x `div` y) * y + (x `mod` y) == x
  where
    nonZero = arbitrary `suchThat` (/= 0)
```

When to prefer `forAll` over `==>`

`==>` is simple but discards many inputs when the precondition is rare, producing fewer effective tests. `forAll` with a custom generator produces only inputs that satisfy the precondition from the start, so all 100 test runs are fully effective.

Coverage and Common Pitfalls

11

QuickCheck can silently pass a flawed test suite. Know the traps.

Vacuous properties

A property with `==>` that discards 99% of inputs is nearly useless. Use `classify` to see what fraction reaches the body.

```
classify (length xs > 10)
  "large list" $
  myProp xs
```

Skewed distributions

`arbitrary for Int` generates values near zero. Use `choose` or `resize` to reach the input space you care about.

```
-- Force larger values
forall (resize 1000 arbitrary)
  myProp
```

Incomplete coverage

100 passing tests is not a proof. Use `cover` to assert at least N% of inputs hit a particular branch.

```
cover 20 (null xs)
  "empty list" $
  myProp xs
```

Recap: QuickCheck as Executable Documentation

12

1. Properties are universal claims — stronger than any finite test suite
2. QuickCheck generates inputs, checks claims, and shrinks failures automatically
3. Arbitrary instances let you control how your types are generated
4. Typeclass laws become runnable properties, not just documentation
5. Smart-constructor invariants (Deck 1) are verifiable with `forAll`
6. `classify` and `cover` make coverage gaps visible, not hidden

Properties are the executable form of the invariants you defined in previous slide deck

